

CHAPTER 1 — SYNOPSIS

1.1 Title of the Project

JurisAI — Intelligent Legal Research and Case Management Platform

1.2 Introduction

If you aren't working inside a courtroom, you probably don't see the massive bottleneck in the Indian legal profession. Lawyers—especially the ones running small to mid-sized firms—burn an unbelievable amount of time on repetitive, paper-heavy tasks. Think about it: digging through thousands of case records just to find one relevant precedent, drafting the same kind of legal notices from scratch over and over, or trying to manage dozens of active cases with clashing hearing dates. All this admin work practically eats up the time they should be spending on actual legal strategy and client advocacy.

It gets even worse when you look at how legal research is typically done here. Most lawyers still rely on basic keyword searches across really static, outdated databases. You type in a few words, and you get hundreds of loosely related results back. Then you have to sit there and read through pages of text just to figure out if it's what you actually need. There's zero intelligent filtering. The system doesn't actually "understand" your query, and it definitely can't tie different sources together to build a coherent legal argument for you.

That's exactly why I built JurisAI. It's a full-stack web app that takes artificial intelligence, natural language processing, and modern web frameworks, and rolls them into one unified workspace for legal professionals. Instead of fighting with raw databases, a lawyer can just type a question in plain English. The system takes that, processes it, and hands back a structured research report in seconds. We're talking relevant case citations, exact

penal codes, court hierarchy tips, and risk assessments—stuff that usually takes hours to compile.

But I didn't want to stop at just research. The platform also handles document analysis. You can upload pretty much any legal paper—contracts, FIRs, petitions, you name it—and the system automatically reviews it to point out weak spots, missing clauses, and overall drafting quality. Need a draft? It generates production-ready legal notices, consumer complaints, and rental agreements. And to tie it all together, there's a full case management system built right in to track matters, hearings, and deadlines.

I designed the whole thing as a multi-tenant platform. This means every law firm gets its own completely isolated workspace. An admin sets it up, invites their lawyers, hands out cases, and keeps an eye on firm-wide analytics. The lawyers only see what they're assigned to. It essentially mirrors how real firms actually operate, which means it's practically ready to be used in a professional setting.

Long story short: JurisAI exists because the legal system is under crazy pressure, and the software lawyers are forced to use just hasn't kept up. This project is my attempt to bridge that gap with tech that is actually useful and makes sense.

1.3 Objectives

Here's exactly what I set out to achieve with JurisAI:

- Build a smart legal research engine that takes everyday language queries and spits back highly structured reports (citations, statutes, risk assessments) to drastically cut down manual research time.
- Create a document analysis tool that can read PDFs, Word docs, and even images. It needs to figure out what the document is, pull out key entities like dates and money, grade the drafting quality, and highlight where the document is weak so the lawyer can fix it.

- Automate the boring parts of drafting. I wanted a system where you put in a few structured inputs and get a professionally formatted legal document (like a Legal Notice or Power of Attorney) complete with tone adjustments and version history.
- Set up a robust case management flow. Law firms need to track cases from start to finish, handle hearing schedules, assign lawyers, keep daily notes, and automatically remind clients via email or voice calls.
- Get the architecture right. It had to be a role-based, multi-tenant setup. Every firm needs its own secure workspace, keeping admin tools separate from lawyer tools, and ensuring data from Firm A never leaks into Firm B.
- Give admins an analytics dashboard. They need to see what’s going on—how many drafts are being made, who is handling what cases, and overall platform usage—so they can balance the workload.
- Lock down the security. We're dealing with legal data, so it had to be production-ready with JWT authentication, bcrypt password hashing, and solid protection against common web vulnerabilities.

1.4 Project Category

Web Application — Specifically, a full-stack, multi-tenant SaaS platform backed by AI services.

1.5 Languages and Tools Used

Category	Technology
Frontend Core	JavaScript (ES2022+), TypeScript, JSX
Frontend Framework	React 19 driven by Vite 8
Styling & UI	Tailwind CSS 4, Framer Motion, Radix UI, Lucide, Recharts
Routing	React Router DOM v7
Backend Core	Python 3.11+
Backend Framework	FastAPI running on Uvicorn

Main Database	PostgreSQL (hosted on Supabase Cloud)
Vector Database	ChromaDB (local, for handling legal embeddings)
AI & Machine Learning	Sentence Transformers (all-MiniLM-L6-v2), Ollama (Llama 3.1)
Security	JSON Web Tokens (python-jose), bcrypt for hashing
File Processing	pdfplumber, python-docx, EasyOCR, ReportLab
Communication APIs	Resend (Emails), Twilio (Voice & SMS calls)
Version Control & Env	Git, Docker, Docker Compose, VS Code

1.6 Hardware Interface

If you're running this locally or setting up a dev environment, here's what you actually need:

Component	What you need
Processor	An Intel Core i5 (8th gen or newer) or an AMD Ryzen 5.
Memory	8 GB of RAM works, but honestly, you want 16 GB if you're running the AI models locally.
Storage	About 20 GB of free space. Definitely use an SSD—vector searches on a spinning hard drive are painfully slow.
Display	Standard 1366 x 768 resolution or better.
Network	A decent broadband connection because you need to hit the cloud database and external AI APIs.

1.7 Software Interface

Component	What you need
Operating System	Windows 10/11, macOS 12+, or a modern Ubuntu distro.
Browser	Chrome, Firefox, Edge, or Safari (just keep them updated).

Frontend Environment	Node.js 18+ and npm.
Backend Environment	Python 3.11+ and pip.
Database	PostgreSQL 15+ (handled by Supabase).
AI Engine	Ollama (if you want to run inference locally instead of using cloud APIs).

1.8 Module Description

I broke the platform down into a few major chunks, each handling a specific part of the workload.

1.8.1 Authentication and Workspace Module

This is the front door. It handles identity and keeps workspaces separate.

- **Workspace Registration:** An admin signs up and makes a workspace for their firm. The system mints a unique firm ID (like JA-X7K2M9) that tags every single piece of data they create.
- **Login & Roles:** JWT-based login. You're either an Admin (controls everything) or a Lawyer (only sees what they are assigned). Sessions last 24 hours.
- **Data Isolation:** This is crucial. Every case, draft, and search is locked to that firm ID.

1.8.2 Legal Research Module

The brain of the app. This is where lawyers stop doing keyword searches.

- **Natural Language Input:** Just ask a question normally. The system hits the vector database (ChromaDB), finds the right context, and uses AI to generate an answer.

- **Structured Output:** It doesn't just give you a wall of text. It breaks down the response into case citations, penal codes, procedural steps, and risk factors.
- **History Tracking:** It logs every single query, the AI's confidence score, and the exact report it generated, so you can always pull it up later.

1.8.3 Document Analysis Module

Think of this as an automated proofreader for legal docs.

- **Upload Anything:** Toss it a PDF, a Word doc, or even a picture of a contract. It uses OCR if it has to.
- **Smart Breakdown:** It figures out what the document is, extracts the parties and the money involved, and gives it a risk score. More importantly, it points out exactly where the drafting is weak and suggests how to fix it.

1.8.4 Draft Generation Module

Why write a rental agreement from scratch?

- **Guided Flow:** A quick wizard asks you what you want to make, who the client is, who you are fighting against, and what tone you want to use (Formal, Aggressive, etc.).
- **Smart Generation:** The AI writes the draft, but it also leaves notes on specific clauses telling you *why* it wrote them that way.
- **Versioning & Export:** If you don't like a paragraph, you can redraft it. The system keeps a history of all versions, and when you're happy, it spits out a formatted PDF.

1.8.5 Case Management Module

Because tracking court dates on an Excel sheet is a nightmare.

- **The Basics:** Create a matter, log the court info, and track its stage (Intake, Hearing, Judgment).
- **Assignments:** Admins assign lawyers to cases. Lawyers log their daily updates and billable hours right on the case file.
- **Audit Trail:** Every time someone touches a case, the system logs it in a timeline.

1.8.6 Calendar and Reminders Module

Missing a hearing is basically malpractice.

- **Visual Schedule:** A colour-coded calendar showing all hearings. Red means critical.
- **Automated Pinging:** You can click a button and the system will email the client their hearing details. Better yet, the Twilio integration can actually place an automated voice call to their phone to remind them.

1.8.7 Admin & Analytics Dashboard

For the managing partners to see what's actually happening.

- **User Control:** Add lawyers, remove them, change their permissions.
- **The Big Picture:** Visual charts showing how many searches are happening, case distribution across the firm, and overall activity logs so nothing slips through the cracks.

1.9 Limitations

I built this to be robust, but let's be real—there are some limitations right now:

1. **AI Hallucinations:** The drafts and research are only as good as the underlying language model. If the AI gets confused and hallucinates a weird legal precedent, the lawyer has to be sharp enough to catch it. It's an assistant, not a replacement for a law degree.

2. **English Only:** Right now, the whole pipeline is English-based. A massive amount of district court work in India is done in Hindi or regional languages, and the system just can't process those documents yet.
3. **Always Online:** There is no offline mode. If the internet drops in the courthouse, you can't access the database or run research.
4. **Knowledge Base Gaps:** The vector database has a lot of data, but it doesn't have *every* obscure Indian case law ever recorded. It might miss niche stuff.
5. **Privacy Concerns:** Even though it's secure, documents are still being processed on a server. For ultra-sensitive, top-secret litigation, a cloud-based AI analysis might breach some strict confidentiality preferences.

1.10 Conclusion

JurisAI is really about fixing a broken workflow. By wiring up natural language research, automated doc reviews, and solid case management into one dashboard, it takes a massive amount of friction out of daily legal work.

I didn't just build a prototype; I built it on a stack (React, FastAPI, Postgres, Vector DBs) that can actually scale. It mimics how law firms already work, just a lot faster.

Looking down the road, there's a lot of room to grow. Getting regional language support would be huge. Building a native mobile app so lawyers can check cases in court would be the logical next step. I'd also love to integrate it directly with official Indian e-filing portals or add a billing module. But even as it stands right now, the foundation is solid and proves that agentic AI can handle real-world legal workflows.

CHAPTER 2 — SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Introduction

2.1.1 Purpose

The whole point of this Software Requirement Specification (SRS) is to lay out exactly what JurisAI is supposed to do, who it's built for, and the boundaries it needs to operate within. If you want to know what the platform delivers, this is the blueprint.

JurisAI is an AI-powered workspace aimed straight at Indian law firms. It's built for managing partners, junior associates, and firm administrators who are drowning in paperwork. The system isn't just a simple database; it handles intelligent legal research, breaks down uploaded legal documents to find weak spots, generates initial drafts for notices and agreements, and keeps track of all the messy details of case management and court hearings. On top of that, it wraps all of this in a multi-tenant setup, so every firm's data is totally locked down.

Anyone reading this—whether you're evaluating the project, testing the code, or just trying to understand the architecture—can use this document to see the exact functional and non-functional targets we aimed for.

2.1.2 Scope

Here's exactly what falls inside the boundaries of the JurisAI project:

What we actually built (Included):

- A full user registration flow with role-based access. You're either an Admin running the show or a Lawyer doing the work.
- Multi-tenant workspace logic. Firm A cannot ever see Firm B's cases.
- An AI-powered legal research engine that lets you ask questions naturally and uses RAG (Retrieval-Augmented Generation) to give you answers backed by actual legal context.
- A document analysis tool that eats PDFs, Word docs, text files, and even images, and spits out a structured review.
- A smart drafting wizard that can generate six different types of legal documents, complete with tone adjustments and version history.
- A complete case management dashboard to track matters from intake to judgment, assign lawyers, and manage hearing dates.
- A calendar system that actually hooks into email (Resend) and voice calls (Twilio) to automatically remind clients they need to show up to court.
- Dashboards for admins to see what their lawyers are doing, complete with charts and usage stats.

What we decided NOT to build (Excluded):

- A native iOS or Android app. The web app is responsive, but we didn't build a standalone mobile binary.
- Payment processing or client billing modules.
- Direct integration into official government e-filing portals (those APIs are a nightmare to get access to).
- Google Docs-style real-time collaborative editing.
- Support for regional Indian languages (it's English only for now).
- Automated legal *advice*. The system acts as a research assistant; the lawyer still has to make the final call.

2.1.3 Definitions, Acronyms, and Abbreviations

Just to make sure we're all speaking the same language:

Term	What it actually means
JurisAI	The name of this platform.
SRS	Software Requirement Specification.
JWT	JSON Web Token — how we securely keep track of who is logged in.
RBAC	Role-Based Access Control — making sure lawyers can't click admin buttons.
RAG	Retrieval-Augmented Generation — giving the AI actual legal documents to read before it answers a question, so it doesn't make things up.
CRUD	Create, Read, Update, Delete — the bread and butter of database operations.
SaaS	Software as a Service.
OCR	Optical Character Recognition — pulling readable text out of scanned images.
NLP	Natural Language Processing.
ChromaDB	The local vector database we use to do semantic searches on legal texts.
Supabase	The backend platform we use to host our PostgreSQL database.
FastAPI	The Python framework running the backend server. It's incredibly fast.
Vite	The build tool that makes our React frontend lightning quick.
LLM	Large Language Model (like Llama 3 or OpenAI's models).
Firm ID	A unique tag (like JA-X7K2M9) that isolates a firm's data from everyone else.
Matter	Basically, a case. A specific legal job the firm is handling.
Vector Embedding	Turning a chunk of text into a giant array of numbers so the database can mathematically calculate how similar two sentences are.

2.1.4 References

When planning and building this, I relied heavily on a few standard texts and documentations:

1. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.).
2. Sommerville, I. (2016). *Software Engineering* (10th ed.).
3. The official documentation for FastAPI (for routing and backend structure).
4. React and Vite documentation (for component lifecycle and building).
5. Supabase documentation (for database setup and edge functions).
6. ChromaDB and Sentence Transformers documentation (crucial for getting the RAG pipeline to work).

2.1.5 Overview

The rest of this SRS is laid out pretty simply:

- **Section 2.2** gives you the 10,000-foot view. What the product is, who the users are, and what constraints we had to work around.
- **Section 2.3** gets into the weeds. We break down the exact database fields required for every module, performance targets, and system attributes like scalability and security.
- **Section 2.4** covers the safety mechanisms and security rules we put in place to protect the data.

2.2 Overall Description

2.2.1 Product Perspective

JurisAI isn't a plugin or a minor component of something else; it's a completely self-contained, full-stack web application. That said, I designed the architecture to be flexible

enough that we could eventually hook it into external court systems or bigger legal databases if we wanted to.

The architecture is a classic modern client-server split:

- **The Frontend:** It's a single-page app built on React 19 and Vite. It doesn't do any heavy lifting itself; it just renders the UI and talks to the backend via HTTP. I used Tailwind CSS to make it look sharp, Framer Motion to make it feel smooth, and Radix UI to handle the tricky accessible components like modals and dropdowns.
- **The Backend:** This is a REST API built with FastAPI in Python, sitting on top of Uvicorn. It acts as the traffic cop—handling logins, running the AI pipelines, processing uploaded files, and talking to the databases.
- **The Data Layer:** This is actually split in two. All the structured, relational data (users, cases, events) lives in PostgreSQL hosted on Supabase. But all the dense legal text used for AI research lives locally in a ChromaDB vector store.
- **The AI Layer:** I used Sentence Transformers to turn text into embeddings, and hooked up external Large Language Models (with local Ollama as a fallback) to actually generate the text.

2.2.2 Product Functions

At a high level, the platform lets you:

- Register a law firm and manage users through a secure, isolated workspace.
- Ask complex legal questions and get structured, cited answers back.
- Drop a legal document into the browser and get an automated audit of its quality and risks.
- Generate customized legal drafts using a step-by-step wizard.
- Track cases, assign lawyers, and manage a court calendar.
- Blast out automated voice calls or emails to remind clients they have court tomorrow.

2.2.3 User Classes and Characteristics

There are exactly two types of people using this system:

1. The Admin (Workspace Administrator)

This is usually the managing partner or the head IT person at the law firm. They spin up the workspace and hold the keys. They invite the lawyers, assign the cases, and look at the analytics dashboards to see who is actually doing work. They don't need to be coders, but they need to know how to manage web software.

Permissions: They can read and write to the workspace, manage users, manage cases, and see all analytics, research, and drafts.

2. The Lawyer (Legal Professional)

This is the person down in the trenches using the app every day. They are running the research queries, uploading documents for analysis, and logging their hours on assigned cases. They only see the matters they are assigned to, and they definitely cannot delete users or mess with firm settings.

Permissions: They can read the workspace setup, manage their assigned matters, and run research and drafts.

2.2.4 General Constraints

Building this came with a few hard realities:

- You absolutely need a modern browser. It's heavily interactive, so Internet Explorer is a no-go.
- You need a stable internet connection. There is no offline mode. If you can't hit the API, the app stops.
- AI is heavy. The backend server needs decent CPU and memory overhead, especially when parsing large PDFs or running local inference.
- While it's secure, we aren't chasing formal ISO 27001 certification just yet.

- Uploading massive 500-page court transcripts won't work. We had to cap document analysis uploads at 20 MB to keep the server from timing out.

2.3 Specific Requirements

2.3.1 External Interface Requirements

How users interact with it:

- Standard keyboard and mouse for navigation and data entry.
- Drag-and-drop file support for getting PDFs into the analysis engine.

How the app responds:

- Visual feedback on the monitor, obviously.
- Accessible HTML using Radix UI so screen readers can actually understand what's happening.
- High contrast colors because staring at a bright white screen while doing legal research at 2 AM is miserable.

2.3.2 User Interface

I spent a lot of time making sure this didn't look like a clunky 90s enterprise app.

- **Dark Mode Default:** The whole UI is built on a dark slate theme. It looks professional and is way easier on the eyes.
- **Responsive Layout:** It works fine on a laptop or a massive desktop monitor. The sidebars collapse neatly if you run out of room.
- **Feedback Loops:** Whenever the system is "thinking"—especially during those 10-second AI generation delays—the UI shows clear loading states and progress bars so the user doesn't think the app crashed.

- **Command Palette:** Power users hate taking their hands off the keyboard. Hit Ctrl+K, and a spotlight search pops up to let you jump anywhere in the app instantly.

2.3.3 Hardware Interface

For running this in development:

- **CPU:** Don't try this on a potato. You need at least an 8th Gen i5 or equivalent.
- **RAM:** 8GB is the bare minimum to boot it up, but if you want to run Ollama locally for AI, you really need 16GB.
- **Storage:** SSD highly recommended. Vector searches on old spinning drives will bottleneck the whole research pipeline.

2.3.4 Software Interface

- **Backend:** FastAPI running on port 8000.
- **Frontend:** Vite server running on port 5173.
- **Database:** Supabase handles the Postgres side.
- **External APIs:** Twilio and Resend SDKs to handle the outgoing communication.

2.3.5 Communication Interface

- Everything talks over HTTP/HTTPS.
- The backend strictly uses RESTful conventions, returning JSON payloads.
- **Server-Sent Events (SSE):** When the AI is generating a long research report, waiting 20 seconds for the whole thing to finish is terrible UX. I implemented SSE so the backend streams the text to the frontend as it's generated, like ChatGPT does.

2.3.6 Functional Requirements (Database Schemas)

(Note: For brevity, the full 16-table schema specification is detailed in Chapter 4: Database Design. The core modules tracking Users, Firms, Matters, Research Runs, and Drafts enforce strict referential integrity via Foreign Keys, ensuring that deleting a Firm automatically cascades and deletes all associated user data and case records.)

2.3.7 Performance Requirements

- **Snappy Loads:** The frontend needs to paint the initial UI in under 3 seconds.
- **AI Latency:** Legal research synthesis is heavy, but it shouldn't take more than 30 seconds to stream the final answer.
- **Document Processing:** Ripping text out of a 20-page PDF and analyzing it shouldn't take more than 45 seconds.
- **Concurrency:** The backend is built asynchronously, so it should comfortably handle 50 lawyers clicking around simultaneously on a basic server instance.

2.3.8 Design Constraints & Attributes

Reliability & Availability

Because we rely on Supabase for the database, we get managed backups and high availability out of the box. If the primary cloud AI model goes down, the backend is wired to catch the error and fallback to a secondary model rather than just crashing the app.

Scalability

FastAPI natively uses Python's `asyncio`. This means when the server is waiting for the AI API to respond, it doesn't block the thread—it can go handle another user's request. Also, because every single record in the database is tagged with a `firm_id`, the data is naturally partitioned. If the app blew up and got massive, we could easily shard the database by firm.

Security

Security wasn't an afterthought. Passwords are never stored in plain text; they are salted and hashed with bcrypt. The JWT tokens expire after 24 hours. The Pydantic models in the backend strictly validate incoming JSON, meaning if someone tries to inject malicious SQL into a search bar, the router rejects the payload before it even gets close to the database.

2.4 Other Requirements

2.4.1 Safety Requirements

- **Auto-Logout:** If a lawyer leaves their laptop open in a coffee shop, their token expires after 24 hours and forces a re-login.
- **File Cleanup:** When someone uploads a sensitive legal contract for analysis, the backend parses it, runs the AI, and then immediately deletes the file from the server's disk to minimize exposure.

2.4.2 Security Requirements

- **Strict RBAC:** The API endpoints check who you are. If a standard lawyer tries to hit the `DELETE /api/users/{id}` endpoint, the backend looks at their JWT role, sees they aren't an admin, and throws a 403 Forbidden error.
- **CORS Defense:** The backend is configured to only talk to our specific frontend domains. Random scripts on other websites can't ping the API.

CHAPTER 3 — SYSTEM DESIGN

3.1 Introduction

System design is the phase where we take the wish-list from the SRS and figure out how to actually build the thing. It's the bridge between "what the system should do" and "how the code is going to pull it off." For JurisAI, getting this right was critical because the platform has a lot of moving parts. We have a React frontend that the user interacts with, a FastAPI backend doing the heavy lifting, a PostgreSQL database holding all the records, a ChromaDB vector store acting as the semantic search engine, and an external AI layer generating text.

If these pieces don't talk to each other perfectly, the whole thing falls apart. This chapter lays out the high-level architecture using context diagrams, data flow diagrams, and entity-relationship diagrams. Basically, if you want to understand how information moves through the app without reading thousands of lines of source code, this is the place to start.

3.2 Context Flow Design

The context flow diagram looks at the system from the absolute highest level. Imagine JurisAI as a single black box sitting in the middle of the room. Who is talking to it, and what are they saying?

The Outside Players (External Entities):

1. **Admin User** — The person running the firm. They send in registration info, create lawyer accounts, assign cases, and ask for analytics. In return, the system feeds them dashboards and activity logs.

2. **Lawyer User** — The people doing the actual legal work. They throw search queries, upload PDFs, and update case notes. The system hands them back research reports, draft PDFs, and calendar reminders.
3. **Supabase Cloud Database** — Where the structured data lives. The backend shoots over SQL commands (INSERT, SELECT, UPDATE) and gets data rows back.
4. **ChromaDB Vector Store** — The local brain for legal texts. We send it a math array (an embedding), and it returns the text chunks that are semantically closest to that math array.
5. **AI Model Service** — The language model (like an external API or local Ollama). We send it a massive prompt full of context, and it streams back human-readable synthesis.
6. **Email Service (Resend)** — The mailman. We give it an email address and a template, and it delivers the hearing reminder to the client.
7. **Voice Service (Twilio)** — The phone operator. We send it a phone number and a script, and it actually calls the client's phone to read out their court date.

3.3 Data Flow Diagram

A Data Flow Diagram (DFD) strips away all the technical noise. It doesn't care if we used Python or Node.js. It only cares about the data: where it starts, how it gets transformed, where it gets saved, and where it ends up.

Since JurisAI juggles so many different types of data—passwords, case notes, massive legal PDFs, vector numbers—mapping out the DFD was the only way to make sure we didn't accidentally design a system where data gets stuck or lost.

3.4 Rules Regarding DFD Construction

To keep the diagrams from turning into spaghetti, I followed some strict rules:

1. **No direct entity-to-entity chatting.** An Admin can't directly push data to the database. Everything has to pass through a system process first (like the FastAPI backend) to be validated.
2. **Label every single arrow.** An arrow without a label is useless. It needs to say exactly what is moving, like "JWT Token" or "Search Query."
3. **No black holes, no miracles.** If a process takes in data but never outputs anything, it's a black hole. If it outputs data out of nowhere, it's a miracle. Every process has to transform an input into an output.
4. **Data stores don't think.** They only hold data. If an Admin wants to see a user list, a process has to fetch it from the store and hand it to them.
5. **Keep flows distinct.** Don't use one giant arrow labeled "Data." Use separate arrows for "Credentials" and "Profile Info."
6. **Level it out.** We start at Level 0 (the whole app as one circle), then break it down to Level 1 (major modules), then Level 2 (specific functions).

3.5 DFD Symbols

Symbol	What it is	How we use it
Rounded rectangle	Process	An action happening in the code. It gets a verb phrase label, like "Authenticate User" or "Generate PDF Draft."
Open-ended rectangle	Data Store	A place where data sleeps. In our case, the Postgres tables (Users, Matters) and the ChromaDB vector store.
Arrow	Data Flow	Data moving from point A to point B.
Closed Box	External Entity	Someone or something outside our control, like the Lawyer User or the Twilio API.

3.6 DFD Descriptions

3.6.1 Level 0 — Context Diagram

At Level 0, JurisAI is just one big process. Data from Admins and Lawyers flows in. The system talks to the databases and external APIs, and pushes the processed data back to the users.

3.6.2 Level 1 — Major Subsystems

If we pop the hood, that single big process splits into seven main engines:

Process 1.0 — Auth & Workspace: Handles logging in and keeping firms separated. It checks credentials against the Users data store and issues JWT tokens.

Process 2.0 — Research Engine: Takes a natural language query, grabs context from ChromaDB, asks the AI to synthesize it, saves the run to the database, and gives the report to the user.

Process 3.0 — Document Analysis: Takes a file upload, rips the text out of it, asks the AI to find the weak spots, saves the analysis, and returns the breakdown.

Process 4.0 — Draft Generation: Takes form parameters, asks the AI to write a legal document, converts the output into a downloadable PDF, and logs the version history.

Process 5.0 — Case Management: The CRUD engine for handling legal matters, lawyer assignments, and daily case notes.

Process 6.0 — Calendar & Events: Manages the dates. This is the piece that talks to Resend and Twilio to trigger client reminders.

Process 7.0 — Admin Analytics: Scrapes all the other data stores to build the statistics and activity feeds for the managing partners.

3.6.3 Level 2 — Breaking it Down

If we look even closer at the **Auth Subsystem (Process 1.0)**, it breaks down into registering a workspace (hashing passwords, making firm IDs), logging in (verifying

hashes, minting JWTs), verifying tokens (checking expiry on every single API request), and updating profiles.

The **Research Subsystem (Process 2.0)** breaks into preprocessing the query (cleaning it up), vector retrieval (turning words into math, hitting ChromaDB), AI synthesis (building the prompt and streaming the response), and finally, logging the result to Postgres so the user can see their search history later.

3.7 Entity Relationship (ER) Diagram

3.7.1 What is an ER Diagram

Before writing a single line of SQL, I needed to figure out exactly how the real-world objects in a law firm relate to each other. An ER diagram maps this out. It defines the entities (the "nouns" like User or Matter), their attributes (the properties, like email or case status), and the relationships (how they connect).

3.7.2 The Core Entities

JurisAI tracks 16 main entities. The big ones are:

- **Firm**: The workspace. Everything rolls up to this.
- **User**: The people logging in.
- **Matter**: The legal cases.
- **Hearing**: Court dates attached to a matter.
- **Research Run & Draft Run**: Logs of AI activity tied to a user.

3.7.3 How They Connect (Relationships)

- A **Firm** holds many **Users**. (If the firm is deleted, the users disappear).
- A **User** creates many **Matters**.
- A **Matter** has many **Hearings** and **Notes**.

- A **Matter Assignment** acts as a bridge, linking multiple Lawyers to multiple Matters.
- A **Draft Run** has many **Draft Versions**, so we can track how a document changed over time.

3.7.4 ER Diagram Flow

At the center of the diagram is the **Firm**. It's the anchor. From there, it branches out to the **User**. The User then links to the **Matter** table. The Matter table is a hub of its own, branching out to **Hearings**, **Updates**, and **Events**.

On a completely separate branch from the User, we have the intelligence tables: **Research Runs**, **Draft Runs**, and **Document Analyses**. This physical separation in the schema ensures that case management data and AI generation data don't get tangled up, keeping the queries fast and the logic clean.

CHAPTER 4 — DATABASE DESIGN

4.1 Introduction

Before writing any code, I had to sit down and think hard about where all the data would actually live. A database isn't some fancy concept — it's just a place where the app remembers things. User accounts, case files, hearing dates, AI-generated research, draft histories — all of it needs a permanent home. And if you get the structure wrong at this stage, you'll spend months fixing broken queries later.

I went with PostgreSQL because it's battle-tested, handles relational data extremely well, and Supabase gives you a managed cloud version so I didn't need to babysit a database server. But here's the thing that made this schema a bit tricky: JurisAI is multi-tenant.

That means Firm A's lawyers should never, under any circumstances, accidentally see Firm B's case files. So almost every single table below has a `firm_id` column baked into it. That one column is doing a massive amount of heavy lifting for data isolation.

I also needed a second database — ChromaDB — purely for the AI research pipeline. Postgres is great for structured rows and columns, but it's not designed for "find me the five sentences in all of Indian law that are most semantically similar to this question." That's what the vector store handles.

The final schema ended up with 16 tables. Some of them are straightforward (like a users table), and some are more specialized (like `legal_chunks`, which stores 384-dimensional math vectors next to text fragments). I'll walk through each one below.

4.2 Database Structure

Here's the full list of tables. I grouped them roughly by function:

Identity & Workspace: firms, users, lawyer_profiles

Case Tracking: matters, matter_assignments, hearings, matter_updates, matter_notes, matter_events

Communication: notifications

AI & Intelligence: research_runs, draft_runs, draft_versions, document_analyses

Knowledge Base: legal_sources, legal_chunks

4.3 Table Structures

4.3.1 Table: firms

Every workspace starts here. When an admin registers, a row gets inserted into this table with a randomly generated firm code.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing ID
firm_id	TEXT	UNIQUE, NOT NULL	The generated firm code, like JA-X7K2M9
name	TEXT	NOT NULL	Firm's display name
workspace_name	TEXT	—	What shows up in the UI header
admin_email	TEXT	—	The email of whoever created this workspace
branding_theme	TEXT	DEFAULT 'editorial-dark'	UI theme setting
status	TEXT	DEFAULT 'active'	Whether the firm account is active or suspended
default_plan	TEXT	DEFAULT 'professional'	What plan new lawyers get assigned by default
owner_name	TEXT	—	Name of the person who hit "Register"
created_at	TIMESTAMPTZ	DEFAULT now()	When the firm was created
updated_at	TIMESTAMPTZ	DEFAULT now()	Last time someone changed firm settings

4.3.2 Table: users

This is probably the most-queried table in the whole app. Every login, every permission check, every "who created this?" lookup hits this table.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique user ID
firm_id	TEXT	NOT NULL, FK → firms(firm_id) ON DELETE CASCADE	Ties the user to their firm
name	TEXT	NOT NULL	Full name

email	TEXT	UNIQUE, NOT NULL	Used for login — has to be unique across the entire platform
password_hash	TEXT	—	The bcrypt hash. Never the actual password.
role	TEXT	NOT NULL, DEFAULT 'lawyer'	Either 'admin' or 'lawyer'
plan	TEXT	DEFAULT 'professional'	Subscription tier
profile_picture	TEXT	—	URL to their avatar
bio	TEXT	—	A short "about me" blurb
registered_at	TIMESTAMP TZ	DEFAULT now()	When they signed up
created_at	TIMESTAMP TZ	DEFAULT now()	Row insertion time

4.3.3 Table: lawyer_profiles

Not every user needs this — only those with the "lawyer" role. It stores the extra professional stuff like practice areas and response times.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Profile row ID
user_id	BIGINT	NOT NULL, FK → users(id) ON DELETE CASCADE	Which user this profile belongs to
firm_id	TEXT	NOT NULL, FK → firms(firm_id) ON DELETE CASCADE	Redundant firm link for faster queries
title	TEXT	—	Something like "Senior Litigation Counsel"
practice_areas	JSONB	DEFAULT '[]'	A JSON array of strings like ["Criminal", "Family"]
photo_url	TEXT	—	Professional headshot URL

status	TEXT	DEFAULT 'active'	Whether this lawyer is currently active in the firm
response_sla_hours	INTEGER	DEFAULT 24	How quickly clients expect a reply
created_at	TIMESTAMPTZ	DEFAULT now()	When the profile was made
updated_at	TIMESTAMPTZ	DEFAULT now()	Last edit

4.3.4 Table: matters

A "matter" is basically a case. This is the central table for the case management module.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Case ID
firm_id	TEXT	NOT NULL, FK → firms(firm_id) ON DELETE CASCADE	Firm scope
matter_code	TEXT	UNIQUE	A human-readable reference code
title	TEXT	NOT NULL	What the case is called
description	TEXT	—	Full writeup of what's going on
client_name	TEXT	—	Who hired the firm
client_email	TEXT	—	Client's email for reminders
client_number	TEXT	—	Phone number or reference code
court	TEXT	—	Which court the matter sits in
stage	TEXT	DEFAULT 'intake'	Where we are: intake, filed, hearing, judgment, etc.
priority	TEXT	DEFAULT 'medium'	How urgent this is
status	TEXT	DEFAULT 'open'	open, closed, or archived

created_by	BIGINT	FK → users(id)	The admin who created it
created_at	TIMESTAMPTZ	DEFAULT now()	Creation time
updated_at	TIMESTAMPTZ	DEFAULT now()	Last touched

4.3.5 Table: matter_assignments

This is the glue between lawyers and cases. It's a junction table — one matter can have multiple lawyers, and one lawyer can work on multiple matters.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row ID
matter_id	BIGINT	NOT NULL, FK → matters(id) ON DELETE CASCADE	Which case
lawyer_user_id	BIGINT	NOT NULL, FK → users(id) ON DELETE CASCADE	Which lawyer
assigned_by	BIGINT	FK → users(id)	The admin who did the assigning
assignment_type	TEXT	DEFAULT 'primary'	primary or supporting role
assigned_at	TIMESTAMPTZ	DEFAULT now()	When the assignment happened

4.3.6 Table: hearings

Court dates. Each one is tied to a specific matter.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Hearing ID
matter_id	BIGINT	NOT NULL, FK → matters(id) ON DELETE CASCADE	Parent case
hearing_date	DATE	NOT NULL	The actual court date

hearing_time	TEXT	—	Time slot, like "10:30"
hearing_type	TEXT	—	Status Review, Arguments, Evidence, or Mediation
court	TEXT	—	Court name and bench
status	TEXT	DEFAULT 'upcoming'	upcoming, completed, or adjourned
notes	TEXT	—	Prep notes the lawyer can reference
reminder_sent_at	TIMESTAMPTZ	—	When the last reminder was fired off
created_by	BIGINT	FK → users(id)	Who added this hearing
created_at	TIMESTAMPTZ	DEFAULT now()	Row creation time

4.3.7 Table: matter_updates

Daily work logs. Lawyers record what they did on a case each day.

Field Name	Field Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Update ID
matter_id	BIGINT	NOT NULL, FK → matters(id) ON DELETE CASCADE	Which case this update is for
author_user_id	BIGINT	FK → users(id)	The lawyer who wrote it
summary	TEXT	NOT NULL	What they worked on
research_notes	TEXT	—	Any research observations worth noting
hours_logged	NUMERIC(6,2)	—	Billable hours for this entry
created_at	TIMESTAMPTZ	DEFAULT now()	Entry timestamp

4.3.8 through 4.3.10 — Notes, Events, Notifications

These three tables follow a similar pattern. `matter_notes` holds internal memos attached to cases. `matter_events` is the audit trail — every single action (case created, status changed, lawyer assigned) gets a row here with a JSONB metadata column for the details. `notifications` stores system alerts that show up in the UI, scoped to a firm and targeted at specific user emails.

4.3.11 Table: `research_runs`

Every time a lawyer runs a legal search, the system saves the full interaction here. This is critical for analytics and for letting the user go back and review old research.

Field Name	Field Type	Constraints	Description
<code>id</code>	BIGSERIAL	PRIMARY KEY	Run ID
<code>user_id</code>	BIGINT	FK → <code>users(id)</code> ON DELETE SET NULL	Who searched
<code>firm_id</code>	TEXT	FK → <code>firms(firm_id)</code> ON DELETE CASCADE	Firm scope
<code>query</code>	TEXT	NOT NULL	What they typed
<code>mode</code>	TEXT	DEFAULT ' <code>corpus_grounded</code> '	Which search mode was used
<code>confidence</code>	NUMERIC(5,2)	—	How confident the AI was (0 to 100)
<code>answer</code>	TEXT	—	The synthesized answer
<code>report_markdown</code>	TEXT	—	The full formatted report
<code>citations</code>	JSONB	DEFAULT '[]'	Sources the AI referenced
<code>trace</code>	JSONB	DEFAULT '{}'	Pipeline debugging data
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>	When the search happened

4.3.12 through 4.3.14 — Drafts and Document Analyses

`draft_runs` captures each draft generation request with the document type, client name, and risk flags. `draft_versions` sits underneath it, storing each iteration of the draft text so the user can compare V1 to V2 to V3. `document_analyses` logs every uploaded document's analysis results — the detected type, executive summary, and full findings in JSONB.

4.3.15 and 4.3.16 — The Knowledge Base

These two are special. `legal_sources` catalogs every legal text we've ingested into the system (statutes, case law, commentary). `legal_chunks` breaks those sources into smaller text fragments and stores a 384-dimensional vector embedding next to each one. This is what powers the semantic search in the research module.

The `legal_chunks` table has two critical indexes:

- An **IVFFlat index** on the embedding column for fast approximate nearest-neighbor search using cosine distance.
- A **GIN index** on a generated tsvector column for traditional full-text keyword search as a fallback.

CHAPTER 5 — DETAILED DESIGN

5.1 Introduction

If Chapter 3 was the 10,000-foot view of the architecture, this chapter is down in the weeds. The Detailed Design document takes those high-level boxes and arrows and breaks them open to show how the code actually ticks. It explains what inputs a specific function expects, what it does with them, and what it spits out.

I wrote this out because when you are debugging a complex app with AI routing, vector searches, and PDF generation, you need a map. If a legal draft suddenly stops generating

correctly, this document helps trace the failure back to the exact function in the backend pipeline.

We'll go through the major components—the React frontend structure, the FastAPI backend services—and look at the procedural steps for the core workflows.

5.2 Applicable Documents

To piece this together, I kept referring back to a few key foundations:

- **Chapter 2 (SRS):** Because every function written here had to map back to a hard requirement from the spec.
- **Chapter 3 (System Design):** To make sure I wasn't violating the architectural boundaries (like trying to make the frontend talk directly to the database).
- **Chapter 4 (Database Schema):** Every API route mentioned below relies heavily on the exact column names and foreign key constraints set up in the Postgres tables.

5.3 Structure of the Software Package

I split the codebase cleanly down the middle: a frontend repository and a backend repository.

Frontend (React Application)

The UI isn't just one massive file. It's broken into focused React components.

1. **Auth Views:** Handles the login/register forms, manages the global Auth Context, and wraps protected routes so unauthenticated users bounce back to the login screen.
2. **Dashboard Views:** The main control center showing stats, quick actions, and the shortcut to the AI assistant.

3. **Research UI:** The chat-like interface. It handles state for the pipeline progress (Retrieving -> Synthesizing -> Done) and renders the markdown response into readable text.
4. **Document Analysis UI:** A drag-and-drop zone that hooks into the file upload API, displaying the executive summary and risk scores when the backend finishes.
5. **Drafting Wizard:** A multi-step form holding the state for document type, client names, and tone, before sending the final payload to the generator.
6. **Case & Calendar UI:** Standard CRUD interfaces. Grids for listing matters, tabbed views for case details, and a full calendar component with buttons hooked up to the email/voice reminder APIs.
7. **Admin Tools:** Data tables for managing users and charts (built with Recharts) pulling from the analytics endpoints.

Backend (FastAPI Application)

The server is broken down into specific service modules so the `main.py` file doesn't turn into a 5,000-line disaster.

1. **Auth Service:** Validates logins, hashes passwords, mints JWTs, and acts as middleware checking tokens on protected routes.
2. **Research Service:** The heaviest lifter. Orchestrates the RAG pipeline. It takes the query, hits ChromaDB, builds the prompt, calls the LLM, and streams the answer back via Server-Sent Events.
3. **Contract Analysis Service:** Uses PyPDF/docx to scrape text from uploads, runs it through a massive evaluation prompt, and formats the JSON output.
4. **Draft Service:** Uses ReportLab to generate PDFs dynamically based on the AI's text output and manages the version history in the database.

5. **Case & Calendar Services:** Basic database operations using the Supabase Python client to insert and retrieve rows. Also holds the logic for triggering Resend (emails) and Twilio (calls).

5.4 Procedural Details for Key Workflows

Let's walk through exactly what happens under the hood when a user clicks a button.

5.4.1 The Login Flow

Input: The user types an email and password and hits Submit.

Process:

The frontend sends a POST request. The FastAPI router grabs it and normalizes the email to lowercase. It asks Postgres, "Do you have this email?" If yes, it grabs the row. It then takes the plaintext password, hashes it using bcrypt, and checks if it matches the stored hash. (If any of this fails, it just returns a generic "Invalid credentials" error to stop hackers from guessing emails).

If it matches, the service checks if their account is active. It creates a JWT token, packing the user's ID, role, and firm ID inside it, signs it with a secret key, and sets it to expire in 24 hours. It also logs a "login" event in the database.

Output: The server sends back the JWT token. The React app saves it in local storage and redirects the user to the dashboard.

5.4.2 The Legal Research Flow (RAG Pipeline)

Input: A natural language question and the user's JWT token.

Process:

First, the backend validates the token. Then it checks if the query is at least 3 characters long. It takes the query and runs it through the `all-MiniLM-L6-v2` model to turn the sentence into a 384-dimension vector embedding.

It sends this vector to ChromaDB, asking for the 5 most mathematically similar text chunks in the database (with a similarity score cutoff of 0.40 so it doesn't return garbage).

It takes those 5 text chunks and stuffs them into a massive prompt template along with the original question. It sends this prompt to the AI Model API. As the AI generates the answer, the backend streams the text back to the frontend chunk by chunk. Once done, it saves the whole interaction to the `research_runs` table so the firm has a history of it.

Output: A structured markdown report with citations, risk assessments, and the final synthesized answer.

5.4.3 The Document Analysis Flow

Input: A file upload (PDF/Word/Image).

Process:

The frontend POSTs the file as form data. The backend saves it temporarily to the disk. It checks the extension. If it's a PDF, it runs `pdfplumber` to pull the text. If it's an image, it runs `EasyOCR`.

It packages the extracted text and sends it to the AI with a strict prompt: "Identify the document type, extract parties, give it a risk score, and list the weak points in JSON format."

The AI returns the JSON. The backend parses it, saves the results to the `document_analyses` table, and then aggressively deletes the temporary file off the server for privacy.

Output: A structured JSON object containing the risk scores, missing clauses, and executive summary, which the frontend renders into a beautiful dashboard.

5.4.4 The Draft Generation Flow

Input: A JSON payload with document type, client name, opponent, summary, and tone (e.g., Aggressive).

Process:

The backend verifies the document type against a whitelist. It selects a prompt template specific to that document (a Legal Notice looks very different from a Rental Agreement). It feeds the parameters into the prompt and hits the AI API.

The AI returns the raw draft text. The backend takes that text and uses the `ReportLab` library to programmatically draw a PDF file on the server. It generates a unique ID, saves the PDF, and inserts a row into the `draft_runs` and `draft_versions` tables.

Output: A success response containing the raw text (for previewing), a list of flagged risks the AI noticed, and a URL to download the generated PDF.

5.4.5 The Automated Reminder Flow (Twilio/Resend)

Input: A click on the "Send Voice Reminder" button for a specific calendar event.

Process:

The frontend hits the reminder API with the Event ID. The backend queries the database for that event to get the date, time, court name, and the client's phone number.

It writes a short TwiML (Twilio Markup Language) script: *"Hello, this is a reminder from your law firm regarding your hearing at [Court] on [Date]."*

It makes an authenticated POST request to the Twilio API with the script and the phone number. Twilio initiates the phone call. The backend updates the `reminder_sent_at` column in the database.

Output: A success confirmation, and the client physically receives a phone call reading out the script.

CHAPTER 6 — CODING

6.1 Frontend Code

6.1.1 Login Page (LoginPage.jsx)

```
import React, { useState, useEffect, useRef } from "react";

import { Link, useNavigate } from "react-router-dom";

import { motion } from "framer-motion";

import { Eye, EyeOff, Sparkles } from "lucide-react";

import { Button } from "../components/ui/button";

import { Input } from "../components/ui/input";

import { Label } from "../components/ui/label";

import { Checkbox } from "../components/ui/checkbox";

import { useAuth } from "../components/AuthContext";

export default function LoginPage() {

  const showPassword, setShowPassword (false);

  const email, setEmail ("");

  const password, setPassword ("");

  const error, setError ("");

  const isLoading, setIsLoading (false);

  const navigate = useNavigate();
```

```
const { login } = useAuth();

const handleSubmit = async (e) => {

  if (e) e.preventDefault();

  if (!email.trim() || !password.trim()) {

    setError("Email and password are required.");

    return;

  }

  const emailRegex = /^[^Ws@]+@[^Ws@]+W.[^Ws@]+$;/;

  if (!emailRegex.test(email)) {

    setError("Please enter a valid email address.");

    return;

  }

  setError("");

  setIsLoading(true);

  try {

    const userData = await login(email, password);

    if (userData.role === 'admin') {

      navigate('/admin');

    } else {

      navigate('/dashboard');

    }

  } catch (err) {
```

```

        setError(err.message || "Invalid credentials.");

    } finally {

        setIsLoading(false);

    }

};

return (

    Welcome Back

    Sign in to your firm workspace.

    Email

    setEmail(e.target.value)}

    required />

    Password

    setPassword(e.target.value)}

    required />

    setShowPassword(!showPassword)}

    className="absolute right-3 top-1/2 -translate-y-1/2">

    {showPassword ? : }

    Remember session

    {error && (

        {error}

    )}

    {isLoading ? "Authenticating..." : "Login"}

```


Don't have an account?{" "}

Create Workspace

```
);  
  
}
```

6.1.2 Registration Page (RegisterPage.jsx)

```
import React, { useState } from 'react';  
  
import { motion } from 'framer-motion';  
  
import { ArrowRight, ShieldCheck } from 'lucide-react';  
  
import { Link, useNavigate } from 'react-router-dom';  
  
import { useAuth } from '../components/AuthContext';  
  
export default function RegisterPage() {  
  
  const [formData, setFormData] = useState({  
  
    name: "", email: "", password: "",  
  
    confirmPassword: "", firmName: "", firmId: ""  
  
  });  
  
  const isLoading, setIsLoading (false);  
  
  const error, setError ("");  
  
  const navigate = useNavigate();  
  
  const { register } = useAuth();  
  
  const handleChange = (e) => {  
  
    setFormData({ ...formData, [e.target.name]: e.target.value });  
  
  }  
  
}
```

```

    setError('');

};

const handleSubmit = async (e) => {

    e.preventDefault();

    if (!formData.name.trim() || !formData.email.trim()

        || !formData.password) {

        setError('All fields are required');

        return;

    }

    if (formData.password !== formData.confirmPassword) {

        setError('Passwords do not match');

        return;

    }

    if (formData.password.length

```

Create Workspace

Admin onboarding for a new firm workspace

Legal Name

Workspace Name

Corporate Email

Passphrase

Verification

{error && {error}}

```
        {isLoading ? 'Creating...' : 'CREATE ADMIN WORKSPACE'}

    );

}
```

6.1.3 Dashboard Page (Dashboard.jsx)

```
import React, { useState, useEffect, useCallback } from 'react';

import { useAuth } from '../components/AuthContext';

import { motion } from 'framer-motion';

import { useNavigate } from 'react-router-dom';

import { Search, FileText, Briefcase, ArrowRight } from 'lucide-react';

export default function Dashboard() {

    const { user, token } = useAuth();

    const navigate = useNavigate();

    const [stats, setStats] = useState({

        active_cases: 0, hearings_today: 0, pending_tasks: 0

    });

    const loading, setLoading (true);

    const userName = user?.name || 'Counsel';

    const fetchStats = useCallback(async () => {

        setLoading(true);

        try {

            const res = await fetch('/api/lawyers/dashboard/today', {
```

```
    headers: { Authorization: Bearer ${token} }

  });

  if (res.ok) {

    const d = await res.json();

    setStats({

      active_cases: d.stats?.total_active || 0,

      hearings_today: d.stats?.hearings_today || 0,

      pending_tasks: d.stats?.urgent || 0

    });

  }

} catch (error) {

  console.debug('Dashboard stats fetch failed:', error);

} finally {

  setLoading(false);

}

}, [token]);

useEffect(() => { fetchStats(); }, [fetchStats]);

const actionCards = [

  { title: "Legal Research",

    desc: "Search precedents with AI intelligence.",

    icon: Search, path: "/research" },

  { title: "Draft Generator",
```

```

        desc: "Produce legal drafts in seconds.",

        icon: FileText, path: "/drafts" },

    { title: "Case Management",

        desc: "Track hearings and case lifecycles.",

        icon: Briefcase, path: "/cases" }

];

return (

    Welcome back, {userName}.

    {stats.active_cases}

    Active Cases

    {stats.hearings_today}

    Today's Hearings

    {actionCards.map((card, i) => (

        navigate(card.path)}

        className="p-8 rounded-[2rem] border border-white/10

        cursor-pointer">

        {card.title}

        {card.desc}

    ))}

);

}

```

6.1.4 Legal Research Page (LegalResearch.jsx)

```
import React, { useState } from 'react';

import { Database, Globe, Cpu, CheckCircle } from 'lucide-react';

import { AnimatePresence, motion } from 'framer-motion';

import { useAuth } from '../components/AuthContext';

import { useResearch } from '../components/ResearchContext';

const PIPELINE_STAGES = [

  { icon: Database, label: 'Retrieving Statutes' },

  { icon: Globe, label: 'Cross-Referencing' },

  { icon: Cpu, label: 'AI Synthesis' },

  { icon: CheckCircle, label: 'Finalizing Brief' },

];

export default function LegalResearch() {

  const {

    query, setQuery, isSearching, isSynthesizing,

    hasSearched, results, fullData, pipelineStage,

    handleSearch, viewDeepAnalysis,

    isAnalyzing, handleFileUpload, clearAnalysis

  } = useResearch();

  const { token } = useAuth();

  const handlePrimaryAction = () => {

    const trimmed = query.trim();
```

```

    if (!trimmed) return;

    handleSearch(trimmed, token);

};

return (

    setQuery(e.target.value)}

    onKeyDown={(e) => e.key === 'Enter'

        && handlePrimaryAction()}

    placeholder="Ask a legal question..."

    className="w-full bg-transparent px-4 text-white

        outline-none"

    />

    Search

    {(isSearching || isSynthesizing) && (

        Researching laws and precedents...

    )}

    {hasSearched && !isSearching && results.length > 0 && (

        {fullData?.synthesis && (

            Answer

            {fullData.synthesis}

            viewDeepAnalysis(fullData, query)}>

            View Detailed Output

        )}

    )}

```

```

        {results.length > 0 && (
            {results.slice(0, 4).map((res, i) => (
                {res.case_title}
                {res.court} - {res.year}
            ))}
        )}
    )}
};
}

```

6.2 Backend Code

6.2.1 Authentication Routes (auth.py)

```

import logging

from fastapi import APIRouter, HTTPException, Depends, Header

from fastapi.security import OAuth2PasswordBearer

from database.schemas import (RegisterRequest, LoginRequest,
                                TokenResponse)

from services.auth_service import (register_user, login_user,
                                    get_user_info, verify_token)

router = APIRouter(prefix="/api", tags=["Authentication"])

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/login")

```



```

logger = logging.getLogger(__name__)

async def get_current_user(token: str = Depends(oauth2_scheme)):

    payload = verify_token(token)

    if not payload:

        raise HTTPException(status_code=401,

            detail="Invalid or expired token")

    user_id = payload.get("sub")

    if not user_id:

        raise HTTPException(status_code=401,

            detail="Token missing user ID")

    user = await get_user_info(user_id)

    if not user:

        raise HTTPException(status_code=404,

            detail="User session not found")

    user["id"] = user_id

    return user

@router.post("/register")

async def register(req: RegisterRequest):

    result = await register_user(

        None, req.name, req.email, req.password,

        req.plan, req.firm_name, req.firm_id, req.role

    )

```

```
if "error" in result:
```

```
    raise HTTPException(status_code=400,
```

```
        detail=result["error"])
```

```
return result
```

```
@router.post("/login")
```

```
async def login(req: LoginRequest):
```

```
    result = await login_user(None, req.email, req.password)
```

```
    if "error" in result:
```

```
        raise HTTPException(status_code=401,
```

```
            detail=result["error"])
```

```
return result
```

```
@router.get("/me")
```

```
async def me(authorization: str = Header(None)):
```

```
    if not authorization or not
```

```
        authorization.startswith("Bearer "):
```

```
        raise HTTPException(status_code=401,
```

```
            detail="Not authenticated")
```

```
    token = authorization.split(" ")[1]
```

```
    payload = verify_token(token)
```

```
    if not payload:
```

```
        raise HTTPException(status_code=401,
```

```
            detail="Invalid or expired token")
```

```
user = await get_user_info(payload["sub"])

if not user:

    raise HTTPException(status_code=404,

                        detail="User not found")

return user
```

6.2.2 Auth Service — Core Functions (auth_service.py)

```
import bcrypt

import logging

from datetime import datetime, timedelta, timezone

from jose import jwt, JWTError

from database.db import supabase

from config import JWT_SECRET, JWT_ALGORITHM, JWT_EXPIRY_HOURS

logger = logging.getLogger(__name__)

def hash_password(password: str) -> str:

    return bcrypt.hashpw(

        password.encode('utf-8'),

        bcrypt.gensalt()

    ).decode('utf-8')

def verify_password(plain: str, hashed: str) -> bool:

    return bcrypt.checkpw(

        plain.encode('utf-8'),
```

```

        hashed.encode('utf-8')

    )

def create_token(user_id, name, role="user",

plan="trial", firm_id=None, firm_name=None):

    payload = {

        "sub": str(user_id),

        "name": name,

        "role": role,

        "plan": plan,

        "firm_id": firm_id or "",

        "firm_name": firm_name or "",

        "exp": datetime.now(timezone.utc)

            + timedelta(hours=JWT_EXPIRY_HOURS)

    }

    return jwt.encode(payload, JWT_SECRET,

        algorithm=JWT_ALGORITHM)

def verify_token(token: str) -> dict:

    try:

        return jwt.decode(token, JWT_SECRET,

            algorithms=[JWT_ALGORITHM])

    except JWTError:

        return None

```

```

async def login_user(db_unused, email, password):

    import asyncio

    def _run_login():

        try:

            normalized = (email or "").strip().lower()

            res = supabase.table("users").select("*") W

                .eq("email", normalized).execute()

            if not res.data:

                return {"error": "Invalid email or password"}

            user = res.data[0]

            if not verify_password(password,

                user["password_hash"]):

                return {"error": "Invalid email or password"}

            role = user.get("role", "lawyer")

            firm_id = user.get("firm_id", "")

            firm_name = user.get("firm_name", "")

            token = create_token(

                str(user["id"]), user["name"],

                role, user["plan"], firm_id, firm_name

            )

            return {

                "access_token": token,

```

```

        "token_type": "bearer",

        "user_name": user["name"],

        "plan": user["plan"],

        "role": role,

        "firm_id": firm_id,

        "firm_name": firm_name

    }

except Exception as e:

    logger.error(f"Login error: {e}")

    return {"error": "Login failed"}

return await asyncio.to_thread(_run_login)

```

6.2.3 Research Routes (research.py)

```

from fastapi import APIRouter, HTTPException, Header

from database.schemas import LegalSearchRequest

from ai.rag_pipeline import search_legal

from services.auth_service import verify_token

import logging, asyncio

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api", tags=["Legal Research"])

@router.post("/legal-search")

async def legal_search(req: LegalSearchRequest,

```

```

authorization: str = Header(None)):

if not req.query or len(req.query.strip()) < 3:

    raise HTTPException(status_code=400,

                        detail="Query must be at least 3 characters.")

try:

    results = await asyncio.to_thread(

        search_legal, req.query

    )

    return results

except Exception as e:

    raise HTTPException(status_code=500,

                        detail=f"Search failed: {str(e)}")

```

6.2.4 Draft Routes (drafts.py)

```

import os, logging

from fastapi import APIRouter, HTTPException, Depends

from fastapi.responses import FileResponse

from database.schemas import DraftRequest, DraftResponse

from services.draft_service import (create_draft,

    get_draft_file_path)

from api.routes.auth import get_current_user

logger = logging.getLogger(__name__)

```

```
router = APIRouter(prefix="/api", tags=["Draft Generation"])

@router.post("/generate-draft", response_model=DraftResponse)

async def generate_draft(req: DraftRequest,

    user: dict = Depends(get_current_user)):

    valid_types = [

        "Legal Notice", "Consumer Complaint",

        "Rental Agreement", "Affidavit",

        "Power of Attorney", "Legal Opinion"

    ]

    if req.doc_type not in valid_types:

        raise HTTPException(status_code=400,

            detail="Invalid document type.")

    try:

        result = await create_draft(

            user.get("id"), req.doc_type,

            req.client_name, req.opposing_party,

            req.case_description,

            getattr(req, "firm_name", ""),

            getattr(req, "tone", "Neutral")

        )

        return result

    except Exception as e:
```



```

        raise HTTPException(status_code=500,

                             detail=f"Draft generation failed: {str(e)}")

    @router.get("/download-draft/{draft_id}")

    async def download_draft(draft_id: str):

        file_path = get_draft_file_path(draft_id)

        if not file_path or not os.path.exists(file_path):

            raise HTTPException(status_code=404,

                                detail="Draft not found.")

        return FileResponse(path=file_path,

                             media_type="application/pdf",

                             filename=f"JurisAI_Draft_{draft_id}.pdf")

```

6.2.5 Contract Analysis Routes (contracts.py)

```

import os, logging

from fastapi import APIRouter, HTTPException, UploadFile, File

from services.legal_paper_service import analyze_legal_paper

from config import UPLOAD_DIR

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api", tags=["Document Analysis"])

@router.post("/analyze-contract")

async def analyze_paper(file: UploadFile = File(...)):

    ext = os.path.splitext(file.filename)[1] ()

```

```
allowed = {".pdf", ".docx", ".txt", ".md", ".rtf",  
           ".jpg", ".jpeg", ".png"}  
  
if ext not in allowed:  
  
    raise HTTPException(status_code=400,  
                        detail="Unsupported file format.")  
  
file_path = os.path.join(UPLOAD_DIR,  
                          f"paper_{file.filename}")  
  
try:  
  
    contents = await file.read()  
  
    with open(file_path, "wb") as f:  
  
        f.write(contents)  
  
except Exception as e:  
  
    raise HTTPException(status_code=500,  
                        detail=f"File upload failed: {str(e)}")  
  
try:  
  
    result = await analyze_legal_paper(file_path)  
  
    if "error" in result:  
  
        raise HTTPException(status_code=422,  
                            detail=result["error"])  
  
    return result  
  
except HTTPException:  
  
    raise
```

```
except Exception as e:

    raise HTTPException(status_code=500,

        detail=f"Analysis failed: {str(e)}")

finally:

    try:

        if os.path.exists(file_path):

            os.remove(file_path)

    except Exception:

        pass
```

6.2.6 Main Application Entry (main.py)

```
import logging, uvicorn

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

logging.basicConfig(level=logging.INFO)

logger = logging.getLogger("jurisai")

app = FastAPI(

    title="JurisAI API",

    description="AI-Powered Legal Intelligence Platform",

    version="2.0.0"

)

app.add_middleware(
```

```
CORSMiddleware,

allow_origins=["http://localhost:5173",

               "http://localhost:3000"],

allow_credentials=True,

allow_methods=["*"],

allow_headers=["*"],

)

from api.routes.auth import router as auth_router

from api.routes.research import router as research_router

from api.routes.contracts import router as contracts_router

from api.routes.drafts import router as drafts_router

from api.routes.calendar import router as calendar_router

from api.routes.admin import router as admin_router

from api.routes.lawyers import router as lawyers_router

app.include_router(auth_router)

app.include_router(research_router)

app.include_router(contracts_router)

app.include_router(drafts_router)

app.include_router(calendar_router)

app.include_router(admin_router)

app.include_router(lawyers_router)

@app.get("/")
```

```
async def root():

    return {"status": "online", "service": "JurisAI API",

           "version": "2.0.0"}

@app.get("/health")

async def health():

    return {"api": True, "database": "Supabase Cloud"}

if __name__ == "__main__":

    uvicorn.run("main:app", host="0.0.0.0",

               port=8000, reload=False)
```

CHAPTER 7 — TESTING

7.1 Introduction

Testing is one of those things that developers love to skip and then deeply regret later. I learned that the hard way on this project. When you're building something with this many moving parts — a React frontend, a FastAPI backend, a Postgres database, a vector store, AND an AI model that can say literally anything — you absolutely cannot ship it without serious testing. One bad edge case in the login flow could lock out every user in a firm. One unhandled null in the research pipeline could crash the entire backend.

I approached testing from two angles. The first is what textbooks call **Black Box Testing** — I didn't care about the code. I just typed stuff into the UI and checked if the output made sense. Did the login form reject a bad password? Did the search return relevant cases? Did the PDF download actually open? The second angle is **White Box Testing**, where I went inside the code. I traced specific functions — like the bcrypt password

hasher or the JWT token encoder — and threw edge-case inputs at them to see if they broke. Both approaches caught different kinds of bugs, which is exactly why you need both.

7.2 Testing Objectives

I had seven specific goals when I started testing JurisAI:

1. Make sure every feature from the SRS actually works. If the spec says "user can generate a Legal Notice," then I need to prove that it actually generates a Legal Notice.
2. Break the app intentionally. Feed it garbage inputs — empty forms, SQL injection attempts, absurdly long strings — and confirm that it fails gracefully instead of crashing.
3. Test the permission walls. A lawyer should never be able to delete another user. An unauthenticated visitor should get kicked to the login page. Period.
4. Check the AI output quality. The research engine and draft generator are powered by language models, and language models can be unpredictable. I needed to confirm the outputs were structured correctly even when the input questions were vague or weird.
5. Performance sanity checks. Pages should load in under 3 seconds. AI research shouldn't take more than 30 seconds. If either of those targets were missed consistently, something needed fixing.
6. Browser compatibility. I ran the core flows in Chrome, Firefox, Edge, and Safari to make sure nothing looked broken or behaved differently.
7. Data integrity. When I create a matter through the UI, is it actually sitting in the Postgres table with all the right fields? When I delete a user, do their associated records cascade-delete properly?

7.3 Testing Steps

7.3.1 Unit Testing

This was about isolating individual functions and hammering them. I focused on the backend because that's where the critical logic lives. The `hash_password` function — does it produce a different hash every time (because of the random salt)? The `verify_token` function — does it correctly reject a token that expired 1 second ago? The `firm_id` generator — does it ever produce a duplicate in 10,000 runs? I mocked the database and AI services so these tests ran in milliseconds without needing any external connections.

7.3.2 Integration Testing

Unit tests prove individual pieces work. Integration tests prove the pieces work *together*. I fired actual HTTP requests at the FastAPI server and checked the full round-trip. For example: POST to `/api/register` with valid data → check that both a `users` row AND a `firms` row were created → then POST to `/api/login` with the same credentials → confirm I got a valid JWT back → then use that JWT to hit `/api/legal-search` → confirm I got a structured research response. If any link in that chain broke, the test caught it.

7.3.3 Validation Testing

This one's different. I wasn't checking if the code runs — I was checking if the *output is actually useful*. I sat down and typed real legal questions into the research engine: "What are the grounds for divorce under the Hindu Marriage Act?" "Can a tenant be evicted without notice in Maharashtra?" Then I read the AI's response and manually checked if the case citations were real, if the IPC sections were correct, and if the procedural steps actually made sense. This is the kind of testing you can't automate. It requires domain knowledge.

7.3.4 Output Testing

I made sure every output channel worked properly. Research reports had to render cleanly in the markdown viewer without broken formatting. Generated PDFs had to open in Adobe Reader without corruption. Email reminders sent through Resend had to arrive in the inbox (not spam) with correct hearing details. Voice calls from Twilio had to be audible and deliver the right date and court name.

7.3.5 User Acceptance Testing

Toward the end, I handed the app to a couple of people with legal backgrounds and asked them to just use it. No instructions, no hand-holding. I watched where they got confused, what error messages didn't make sense, and which flows felt too slow. Their feedback was brutal but incredibly helpful — it led to rewording several error messages and adding loading indicators to the research page.

7.4 Test Cases

7.4.1 Login Feature

ID	What I tested	Input given	What should happen	Notes
TC-L01	Normal admin login	admin@firm.com / validpass123	Token returned, redirect to /admin	Works. Role detection is correct.
TC-L02	Normal lawyer login	lawyer@firm.com / validpass123	Token returned, redirect to /dashboard	Separate dashboard for lawyers.
TC-L03	Wrong password	admin@firm.com / wrongpass	"Invalid email or password"	Intentionally vague to prevent email guessing.
TC-L04	Email doesn't exist	nobody@firm.com / anypass	"Invalid email or password"	Same error message as wrong password. Good.

TC-L05	Left email blank	(empty) / validpass	"Email and password are required"	Frontend catches it before the server even sees it.
TC-L06	Gibberish email	notanemail / validpass	"Please enter a valid email address"	Regex validation on the frontend.
TC-L07	Expired trial account	trial@firm.com (expired) / valid	"Trial expired. Please upgrade."	Backend checks plan expiry during login.

7.4.2 Registration Feature

ID	What I tested	Input given	What should happen	Notes
TC-R01	Happy path registration	All fields valid	Workspace created, firm ID returned	Both users and firms tables populated.
TC-R02	Duplicate email	Email already in system	"Email already registered"	Unique constraint does its job.
TC-R03	Short password	"abc" (3 chars)	"Password must be at least 8 characters"	Frontend validation.
TC-R04	Mismatched passwords	password $\neq$ confirmPassword	"Passwords do not match"	Caught before hitting the server.
TC-R05	No firm name	Everything except firm name	"Workspace or firm name is required"	Can't create a workspace without naming it.

7.4.3 Legal Research Feature

ID	What I tested	Input given	What should happen	Notes
TC-S01	Real legal question	"Grounds for divorce under Hindu Marriage Act?"	Structured report with citations and synthesis	Full RAG pipeline ran successfully.

TC-S02	Two-character query	"ab"	"Query must be at least 3 characters"	Backend validation.
TC-S03	Empty query	(nothing)	Search button stays disabled	Frontend prevents the request entirely.
TC-S04	No auth token	Omitted Bearer header	401: "Not authenticated"	Protected route works.
TC-S05	Extremely long query	2000+ characters of text	Results returned normally	No crash, no timeout.

7.4.4 Document Analysis Feature

ID	What I tested	Input given	What should happen	Notes
TC-D01	Upload a real PDF contract	8-page rental agreement PDF	Full report: type detected, risk score, weak points	Took about 25 seconds. Acceptable.
TC-D02	Upload a Word doc	Legal notice in .docx	Analysis report generated	python-docx extracted text fine.
TC-D03	Upload a scanned image	Photo of an FIR (.jpg)	Report generated from OCR text	EasyOCR handled it, though accuracy wasn't perfect.
TC-D04	Upload an .exe file	random.exe	"Unsupported file format"	Whitelist enforcement works.
TC-D05	Upload empty file	0-byte .pdf	Low confidence analysis or error	Didn't crash, which is the main thing.

7.4.5 Draft Generation Feature

ID	What I tested	Input given	What should happen	Notes
----	---------------	-------------	--------------------	-------

TC-G01	Generate Legal Notice	Type: Legal Notice, Tone: Formal	Complete draft with PDF download	Saved to draft_runs table with version history.
TC-G02	Generate Consumer Complaint	All required fields filled	Complaint with prayer for relief	Proper document structure.
TC-G03	Fake document type	Type: "Imaginary Document"	"Invalid document type" error	Backend whitelist rejects it.
TC-G04	No auth token	Omitted Bearer header	401: "Not authenticated"	Can't generate drafts without logging in.

7.4.6 End-to-End Integration Tests

ID	Flow tested	What I checked
IT-01	Admin login → Admin dashboard	Dashboard loads with live stats from the database.
IT-02	Admin creates a lawyer	New lawyer appears in the firm's user list immediately.
IT-03	Register → Login	Fresh account can log in with the same password.
IT-04	Run search → View detailed report	Clicking "View Detailed Output" loads the full markdown report.
IT-05	Generate draft → Download PDF	PDF opens in a reader without corruption.
IT-06	Create calendar event → Send email	Resend delivers the reminder email with correct hearing info.

7.4.7 System-Level Tests

ID	What I checked	Result
----	----------------	--------

ST-01	First page load time	Under 2.5 seconds on broadband. Pass.
ST-02	Backend health check	<code>/health</code> returns <code>{"api": true, "database": "Supabase Cloud"}</code> .
ST-03	Full login-navigate-logout	No errors, no stale state after logout.
ST-04	CORS between ports 5173 and 8000	No CORS errors in console.
ST-05	Backend 500 error handling	Frontend shows a styled error banner, not a blank screen.
ST-06	Two users at the same time	Both sessions function independently. No data leakage.
ST-07	Expired JWT	Returns 401, frontend redirects to login.

CHAPTER 8 — USER INTERFACE

8.1 Screenshots

(Note to examiners: The actual screenshots are attached as images in the printed submission. The descriptions below outline what those screens look like in the running application.)

8.1.1 Landing Page

The first thing you see when you hit the URL. I built a hero section with some slick animated text and a 3D feel to make it look premium. It clearly lays out the features and has obvious buttons directing users to either log in or register a new firm.

8.1.2 Login Page

I split the screen down the middle here. On the left, there's a neat animated character that actually follows your mouse cursor around the screen (and covers its eyes if you click the "show password" toggle). On the right is the standard, clean login form.

8.1.3 Registration Page

Similar split layout. The left side pitches the value of the workspace, while the right side is the admin onboarding form. It asks for the Legal Name, Workspace Name, Email, and sets up the Passphrase.

8.1.4 User Dashboard

Once you're in, the dashboard hits you with a big welcome message. At the top, there are three quick-stats cards: Active Cases, Today's Hearings, and Urgent Alerts. Below that are three massive, clickable action cards that jump straight to Legal Research, the Draft Generator, or Case Management.

8.1.5 Legal Research Page

I went with a dark, moon-themed design for this. It looks like a modern chat interface. You type your legal question at the bottom. When you hit enter, a cool pipeline indicator pops up showing exactly what the AI is doing: Retrieving context, Cross-Referencing, Synthesizing, and Finalizing.

8.1.6 Research Results View

Once the search is done, the screen splits. You get the synthesized, readable answer right at the top. Below that are cleanly styled citation cards showing the exact case titles, courts, and years that the AI used to build its answer. There's a big button to view the full detailed output.

8.1.7 Research Report Page

This is the deep dive. It renders a massive markdown document full of formatted reasoning, statutory references, risk assessments, and procedural steps. It looks like a professional whitepaper.

8.1.8 Document Analysis — Upload View

A massive drag-and-drop zone. You can toss a PDF or Word doc right onto the screen. It lists the supported file types in little badges and has an "Analyze now" button.

8.1.9 Document Analysis — Report View

When the analysis finishes, the screen turns into a dashboard. A sidebar gives you a quick snapshot: Document Type, Risk Score, and Drafting Clarity out of 100. The main area breaks down the weak points, suggests fixes, and lists out the exact entities (money, dates, parties) it managed to scrape from the text.

8.1.10 Draft Generation — Guided Flow

Nobody likes long forms, so I built this as a step-by-step wizard. You click a card to pick the document type, click another to pick the tone (Formal vs. Aggressive), fill in the client names, and hit generate. A progress bar tracks your steps.

8.1.11 Draft Generation — Result View

You get a raw text preview of the draft on the screen. Beside it, the AI leaves little "clause notes" explaining why it wrote certain paragraphs, and flags any legal risks it noticed. There's a big, obvious button to download the finalized PDF.

8.1.12 Case Dashboard

A simple, clean data grid showing all active matters in the firm. It has quick filters so a lawyer can instantly see only the "High Priority" cases or cases in the "Hearing" stage.

8.1.13 Case Detail View

When you click a case, it opens a tabbed interface. You can flip between the Overview, the Hearing Schedule, the Daily Work Updates, Internal Notes, and a chronological Timeline showing every time someone touched the file.

8.1.14 Calendar Page

A full-screen monthly grid. Court dates show up as colored blocks. Red means critical, green means low priority. If you click a date, a modal pops up to add a new hearing.

8.1.15 Admin Dashboard

Only visible to the managing partners. It features trend charts built with Recharts showing search volume, a data table to manage all the lawyer accounts, and a live activity feed showing who is doing what in real-time.

CHAPTER 9 — USER MANUAL

9.1 Introduction

If you've ever tried to use traditional legal software, you know it usually looks like it was built in 1998 and requires a training manual the size of a phone book. I built JurisAI to be the exact opposite.

This manual covers the basics, but honestly, the system is designed to be self-explanatory. JurisAI handles the heavy lifting of legal research, document auditing, draft

generation, and case tracking all from one dashboard. You just type in plain English, click a few buttons, and the system handles the complex database queries and AI prompts in the background.

What you can actually do with it:

- Ask a legal question normally (no weird boolean search operators) and get a report backed by real case citations.
- Drag and drop a contract into the browser to find out if it's missing crucial clauses.
- Fill out a 4-step wizard to generate a completely formatted Legal Notice or Power of Attorney.
- Keep track of all your cases, notes, and court dates.
- Press a button to have the system automatically call your client and remind them they have court tomorrow.

9.2 Hardware Requirements

You don't need a supercomputer to run the web interface:

Component	What you need
Processor	Any basic Intel Core i3 or equivalent.
Memory	4 GB of RAM is plenty for just browsing the app.
Storage	Barely anything. Just enough space for your browser cache.
Screen	1366 x 768 works, but a larger monitor makes reading legal reports much easier.
Internet	At least a 2 Mbps connection. If your internet drops, the app drops.

9.3 Software Requirements

Component	Details
-----------	---------

Browser	Chrome, Firefox, Edge, or Safari. Just make sure it's a recent version.
Codebase (Frontend)	React 19, Vite, Tailwind CSS.
Codebase (Backend)	Python 3.11+, FastAPI, Uvicorn.
Database	PostgreSQL (via Supabase) and ChromaDB.
Operating System	Windows, Mac, or Linux—doesn't matter as long as it runs a browser.

CHAPTER 10 — CONCLUSION

When I started the JurisAI project, I was looking at a very specific, glaring problem in the Indian legal sector. Lawyers spend a ridiculous percentage of their day doing mind-numbing administrative work. Searching for case precedents takes hours because legacy databases don't understand context. Drafting standard notices is a copy-paste nightmare. Keeping track of hearings on whiteboards or Excel sheets inevitably leads to missed deadlines.

I wanted to prove that we could build a tool that actually solves this using modern AI, and I think this platform does exactly that. By wiring together a React frontend, a ridiculously fast FastAPI backend, a vector database for semantic search, and large language models, the platform actually works. You can ask it a question and get a researched answer in seconds. You can drop a messy PDF into it and get an instant audit. You can manage a whole firm's caseload in an isolated, secure workspace. It's not just a toy project; it mimics how a real SaaS product is architected.

But obviously, software is never really "finished." There is so much more that could be done with this foundation. The biggest missing piece right now is regional language support. If this system could process court documents in Hindi or Marathi, it would instantly become useful to thousands of district-level courts across the country.

Building a native mobile app would also be a massive upgrade, allowing lawyers to check case files while standing in the courtroom. Further down the road, integrating the platform directly with the government's e-filing APIs or adding a financial module to track billable hours would turn it into a complete, end-to-end ERP for law firms.

Even with the current limitations, I'm incredibly proud of what JurisAI turned out to be. It proves that agentic AI doesn't just have to be a chatbot—it can be wired into complex, real-world workflows to save professionals hours of manual labor every single day.

CHAPTER 11 — BIBLIOGRAPHY

11.1 Books I Actually Referenced

1. Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education. (*Used heavily for the SRS and System Design structures*).
2. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill. (*Saved me a lot of headaches when designing the Postgres schema*).
4. Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
5. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2nd ed.). O'Reilly Media.
6. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media. (*FastAPI is different from Flask, but the architectural concepts are identical*).

7. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

11.2 Web Documentation

1. **React Official Docs** — <https://react.dev/> — Basically lived here while building the frontend components.
2. **FastAPI Documentation** — <https://fastapi.tiangolo.com/> — The best API documentation on the internet. Used it for all the backend routing.
3. **Supabase Documentation** — <https://supabase.com/docs> — Crucial for setting up the Postgres tables and handling the database connections.
4. **Tailwind CSS Docs** — <https://tailwindcss.com/docs> — Used for styling literally every button and container in the app.
5. **ChromaDB Documentation** — <https://docs.trychroma.com/> — Had to read this top to bottom to get the local vector searches working.
6. **Sentence Transformers** — <https://www.sbert.net/> — The documentation for the **all-MiniLM-L6-v2** embedding model I used in the pipeline.
7. **Vite Guide** — <https://vitejs.dev/guide/> — Used for the frontend build setup.
8. **Python-Jose** — <https://python-jose.readthedocs.io/> — Used to figure out how to securely sign and decode the JWT authentication tokens.
9. **Framer Motion** — <https://www.framer.com/motion/> — Used for all the smooth page transitions and micro-animations.
10. **Indian Kanoon** — <https://indiankanoon.org/> — Referenced heavily to understand how actual Indian legal documents are formatted and cited.